PROJET FULLSTACK – S8 | APC

Document de Conception

GameBoard — Plateforme de notation de jeux de société

Équipe	Amina SERRANO · Christina LOPES · Hajar KHETTOU · Lyna MEDJEDOUB · Rawane OUFFA
GitHub	https://github.com/hajar-khettou/Fullstack-project
Encadrant	David THIBAU – david.thibau@gmail.com
Livrable	Milestone 1 – 6 Mars 2026
Version	v1.0 – Document de conception initial

1. Vision Fonctionnelle

1.1 Motivation & contexte

Les amateurs de jeux de société manquent d'une plateforme simple pour découvrir, évaluer et partager des avis sur des jeux. Les informations sont dispersées sur des forums ou des sites étrangers.

GameBoard répond à ce besoin en centralisant le catalogue, les fiches détaillées et les notations communautaires en un seul endroit. L'alimentation automatique via l'API BoardGameGeek (BGG) garantit des données fiables sans saisie manuelle.

1.2 Objectifs du projet

- Référencer un catalogue de jeux de société enrichi automatiquement via BGG
- Permettre aux utilisateurs de noter et commenter les jeux
- Permettre aux éditeurs de proposer de nouveaux jeux
- Assurer une modération du catalogue via un rôle Webmaster
- Mettre en place une architecture backend orientée événements avec Apache Kafka
- Industrialiser le projet avec Docker, CI/CD et une documentation complète

1.3 Profils utilisateurs

Profil	Accès	Fonctions principales
Utilisateur	Public	Consulter le catalogue, voir fiche détaillée, rechercher/filtrer, noter un jeu
Éditeur	Authentifié (rôle éditeur)	Proposer un nouveau jeu via un formulaire, enrichissement automatique via BGG API
Webmaster	Authentifié (rôle admin)	Valider/refuser les jeux proposés, modifier/supprimer jeux, gérer les utilisateurs

2. Backlog & User Stories

Le backlog est priorisé selon la méthode MoSCoW.
La colonne MVP ✔ identifie les stories incluses dans la version 1.0.0 à livrer idéalement le 27 mars.
Les stories sans ✔ sont prévues pour la Release finale du 24 avril.

MUST	Indispensable au MVP v1.0.0	**SHOULD**	Important, prévu en Release	**COULD**	Bonus si le temps le permet

2.1 Utilisateur

#	Rôle	Je veux…	Afin de…	Priorité	MVP
US-01	Utilisateur	consulter la liste paginée des jeux accessible sans connexion	découvrir les jeux disponibles	**MUST**	✔
US-02	Utilisateur	voir la fiche détaillée d'un jeu accessible sans connexion	accéder à la description, image, notes	**MUST**	✔
US-03	Utilisateur	noter un jeu (score 1–5 + commentaire) connexion requise pour noter	partager mon avis avec la communauté	**MUST**	✔
US-04	Utilisateur	rechercher/filtrer par nom, genre, année accessible sans connexion	trouver rapidement le jeu souhaité	**MUST**	✔
US-05	Utilisateur	m'authentifier pour accéder aux fonctionnalités protégées connexion requise uniquement pour noter, proposer ou administrer	accéder aux fonctionnalités protégées	**MUST**	✔

2.2 Éditeur

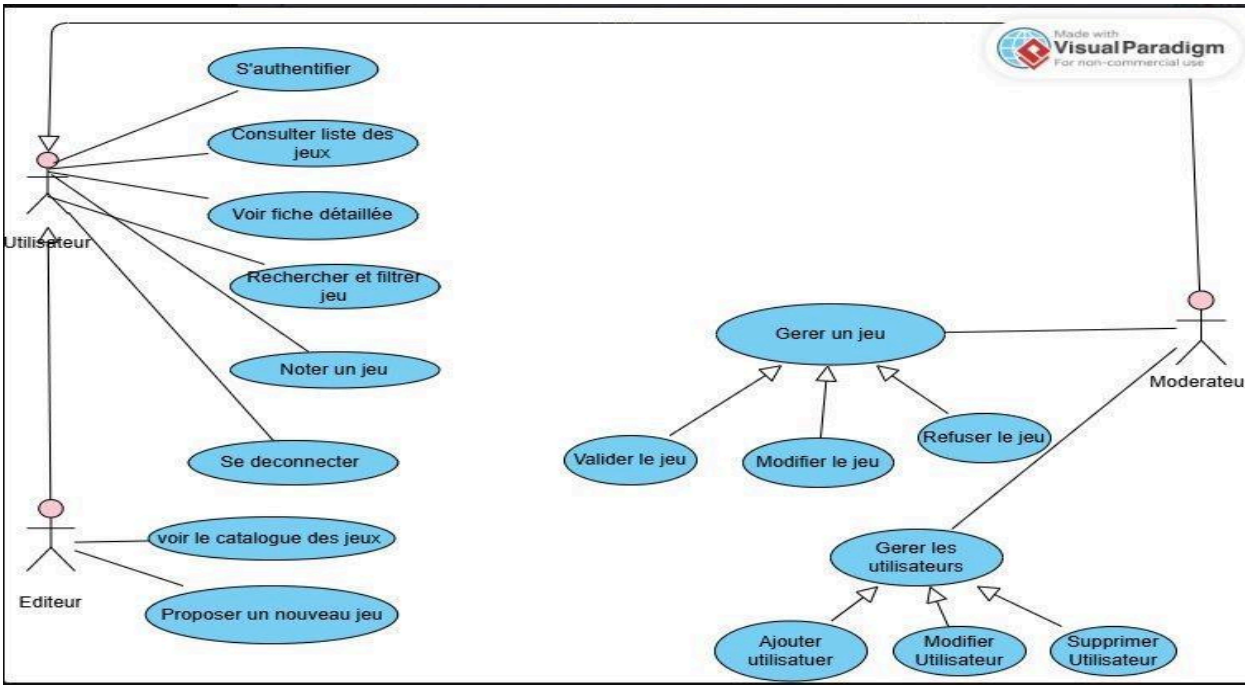
#	Rôle	Je veux…	Afin de…	Priorité	MVP
US-06	Éditeur	proposer un nouveau jeu via formulaire	enrichir le catalogue de la plateforme	**MUST**	✔
US-07	Éditeur	avoir les données auto-complétées depuis BGG	gagner du temps et fiabiliser les infos	**MUST**	✔
US-08	Éditeur	suivre le statut de ma proposition	savoir si mon jeu a été validé ou refusé	**SHOULD**	

2.3 Webmaster

#	Rôle	Je veux…	Afin de…	Priorité	MVP
US-09	Webmaster	valider ou refuser un jeu proposé	garantir la qualité du catalogue	**MUST**	✔
US-10	Webmaster	modifier ou supprimer un jeu existant	maintenir la cohérence du catalogue	**MUST**	✔
US-11	Webmaster	sécuriser les endpoints d'écriture (auth simple, clé API ou Spring Security)	sécuriser les opérations sensibles	**MUST**	✔
US-12	Webmaster	gérer les comptes utilisateurs et éditeurs	administrer la plateforme	**SHOULD**	

3. Diagramme de cas d'utilisation

Le diagramme ci-dessous illustre les interactions entre les trois profils (Utilisateur, Éditeur, Modérateur/Webmaster) et les fonctionnalités de la plateforme.



4. Organisation & Planning

4.1 Méthodologie

Le projet suit une approche Agile Scrum simplifiée avec 4 sprints calés sur les jalons imposés. Chaque sprint inclut les cérémonies agiles suivantes :

- Sprint Planning : définition des tâches et répartition en début de sprint
- Daily asynchrone : mise à jour des issues GitHub
- Sprint Review : démonstration des fonctionnalités réalisées à la fin du sprint
- Rétrospective : identification des points d'amélioration pour le sprint suivant

4.2 Planning des sprints

Sprint	Période	Objectifs	Livrables
Sprint 0(en cours)	19 – 6 mars.	Mise en place de l'environnement DevOps : création du dépôt Git, structure des dossiers, configuration GitHub Actions, docker-compose Kafka + PostgreSQL, conventions d'équipe, document de conception.	Repo Git structuréCI GitHub Actionsdocker-composeDocument de conception v1.0
Sprint 1	26 fév.– 14 mars	API CRUD jeux (Spring Boot), intégration BGG API XML, parsing XML→JSON, Kafka producer, tests unitaires Service/Controller.	API REST fonctionnelleIntégration BGGKafka producerTests unitaires
Sprint 2(MVP)	15 – 27 mars	Kafka consumer, Auth0 RBAC + Spring Security, tests d'intégration, frontend Angular MVP (liste, détail, formulaire), docker-compose complet.	MVP fonctionnelDocumentation READMECouverture ≥ 70%
Sprint 3(Release)	28 mars– 24 avril	Tests e2e, amélioration CI/CD, Swagger/OpenAPI, packaging Docker, dimensionnement prod, préparation soutenance.	Release 1.0CI/CD complèteDémo productionSoutenance

4.3 Conventions d'équipe

- Branches : main (stable, protégé) + feature/xxx (une branche par User Story)
- Commits : format conventionnel (feat:, fix:, docs:)
- Issues GitHub : tâches assignées et trackées
- Communication : groupe WhatsApp/Discord + issues GitHub

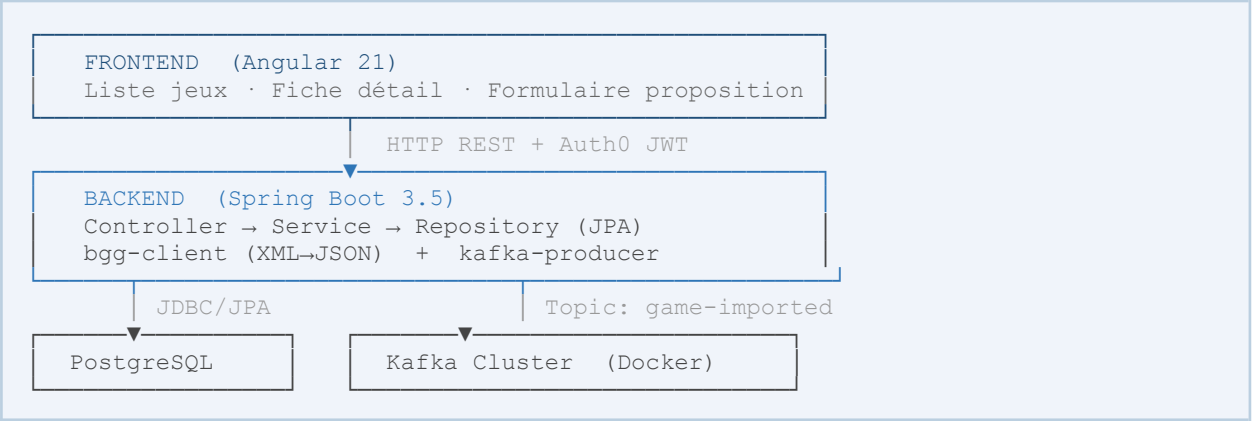
5. Architecture & Stack Technique

5.1 Stack technique choisie

Couche	Technologies retenues	Justification
Backend	Java 21 · Spring Boot 3.5 · Spring Web · Spring Data JPA · Spring Security	Maîtrise de l'équipe, robustesse, écosystème riche
Messaging	Apache Kafka (cluster Docker) · Topic : game-imported	Architecture événementielle, découplage asynchrone
Base de données	PostgreSQL (prod) · H2 (tests)	Fiable, compatible JPA, H2 pour tests légers
Authentification	Sécurité au choix : Auth0, clé API ou Spring Security simple selon la complexité	Gestion des rôles user/editor/webmaster simplifiée
API externe	BoardGameGeek XML API v2	Source de données fiable pour enrichissement automatique
Frontend	Angular 21 · TypeScript · Appels REST	MVP minimaliste : liste, fiche, formulaire
DevOps	Docker · docker-compose · GitHub Actions · Swagger/OpenAPI	Industrialisation et reproductibilité de la stack

5.2 Architecture logicielle

Le backend suit une architecture en couches avec communication événementielle via Kafka :



5.3 Intégration Kafka

Topic	game-imported
Déclencheur	Import d'un jeu depuis BGG (proposition éditeur ou import manuel)
Producteur	kafka-producer — déclenché par la couche Service après appel BGG
Consommateur	kafka-consumer — log structuré + traitement asynchrone
Payload	{ gameId, title, source: 'editor'|'manual', importedAt: ISO-8601 }
Preuve démo	Log console visible en temps réel lors de la démonstration

5.4 Modèle de données

Entité BoardGame

Champ	Type	Description
id	Long (PK)	Identifiant auto-généré
title	String	Titre du jeu (source BGG) — @NotBlank
description	Text	Description longue (source BGG)
imageUrl	String	URL image de couverture (source BGG)
minPlayers	Integer	Nombre minimum de joueurs
maxPlayers	Integer	Nombre maximum de joueurs
year	Integer	Année de publication
averageRating	Double	Moyenne des notes calculée dynamiquement
status	Enum	PENDING (défaut) / APPROVED / REJECTED

Entité Rating

Champ	Type	Description
id	Long (PK)	Identifiant auto-généré
boardGameId	Long (FK)	Référence vers BoardGame
userId	String	Identifiant utilisateur / nullable (anonyme possible)
score	Integer	Note 1 à 5
comment	String	Commentaire libre (optionnel)
createdAt	LocalDateTime	Date/heure automatique à la création

6. Plateforme DevOps

La plateforme DevOps est en cours de mise en place dans le cadre du Sprint 0. Les éléments ci-dessous décrivent les choix techniques retenus et l'état d'avancement.

6.1 Gestion du code source

- Dépôt GitHub : https://github.com/hajar-khettou/Fullstack-project
- Stratégie de branches : main (stable, protégé) + feature/xxx (une par User Story)
- Convention commits : feat: / fix: / test: / chore: / docs:
- Merge Requests : relecture encouragée par un autre membre avant merge sur main

6.2 Pipeline CI — GitHub Actions

Pipeline déclenchée à chaque push et Pull Request sur main :

Étape	Outil	Description
1. Checkout	actions/checkout@v3	Récupération du code source
2. Setup Java	actions/setup-java@v3	Installation Java 21 (Temurin)
3. Build	Maven - mvn clean install	Compilation du projet - échec si erreur
4. Tests	Maven - mvn test	Exécution des tests unitaires et d'intégration

6.3 Containerisation

- docker-compose.yml : orchestre PostgreSQL, Zookeeper et Kafka Broker
- Variables d'environnement externalisées pas de secrets en clair dans le code
- Dockerfile backend : à créer en Sprint 1 (image OpenJDK 21)
- Stack complète lançable en une commande : docker-compose up -d

6.4 Documentation & monitoring

- README : lancement from scratch documenté (prérequis, docker-compose, run, tests)
- Swagger / OpenAPI 3 : documentation des endpoints REST (à générer en Sprint 2)
- Spring Actuator /health : endpoint de santé (inclus dans les dépendances Maven)